

Bien utiliser Claude sans exploser son budget de tokens

Ce qui fait vraiment grimper la consommation, comment structurer les fils de discussion, et une checklist de démarrage pour ton prochain projet de jeu.

Préparé pour Steve — à partir de la documentation officielle Anthropic (Claude Code & Claude Cowork)
Septembre 2026

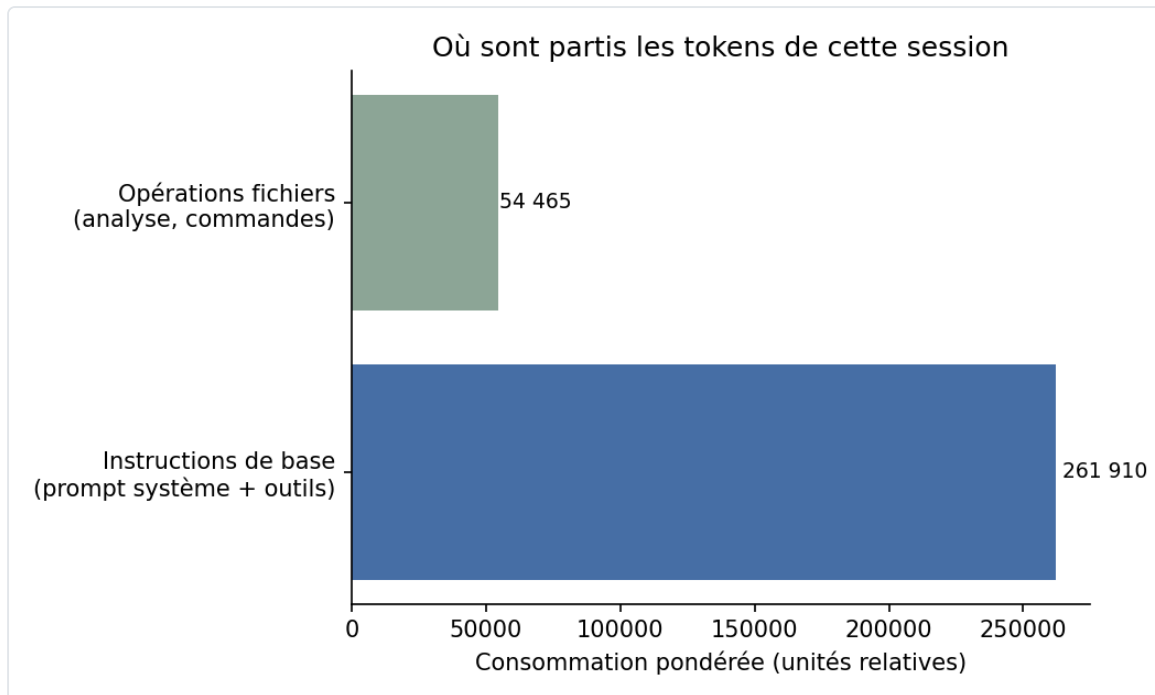
1. Le constat : pourquoi un fil de discussion long coûte de plus en plus cher

Ce n'est pas la longueur de tes demandes qui pose problème — c'est la longueur cumulée de la conversation.

À chaque message que tu envoies, Claude ne reçoit pas que ta dernière phrase : il reçoit **toute la conversation depuis le début du fil** — chaque fichier lu, chaque commande exécutée, chaque réponse précédente. Un mécanisme de cache rend cette relecture beaucoup moins chère (environ 10 fois moins qu'une lecture "à froid"), mais elle n'est jamais gratuite, et elle compte quand même dans ton quota.

Concrètement, deux choses grossissent avec le temps :

- **Le socle fixe** — les instructions système et la liste des outils disponibles. Il se recharge à chaque échange.
- **L'historique** — tout ce qui s'est dit et fait depuis le début du fil, y compris les longs extraits de fichiers ou de code déjà collés.



Exemple mesuré sur cette session : même une toute petite tâche paie un socle fixe important avant la moindre vraie réponse.

CE QUI AGGRAVE ENCORE LES CHOSES

Si un fil reste inactif plus d'une heure, le cache "refroidit" : le message suivant recharge tout l'historique à plein tarif au lieu du tarif réduit. Un fil qu'on laisse ouvert des semaines, qu'on ouvre de temps en temps, est donc le pire des cas — c'est probablement ce qui s'est passé sur le projet en cours : beaucoup de mémoire accumulée (specs, conventions, lots de contenu) suggère un ou deux très longs fils réutilisés sur toute la durée du projet plutôt que des tâches cloisonnées.

2. Les trois leviers propres à Claude Cowork

La bonne nouvelle : Cowork est justement conçu pour éviter ce piège.

Le Projet

Un **Projet** regroupe un dossier connecté, des instructions permanentes et une mémoire — exactement ce que tu as déjà mis en place pour le jeu actuel (instructions Godot, conventions de style, specs d'art). C'est la base persistante : elle ne se recharge pas à chaque message, elle est disponible d'emblée.

La Tâche (le fil de discussion)

Chaque tâche démarre avec son propre contexte de conversation — vide au départ. C'est l'histoire de la tâche qui s'accumule et coûte cher au fil du temps, pas le projet lui-même.

La Mémoire de projet

Point clé souvent ignoré : **la mémoire survit d'une tâche à l'autre, à l'intérieur d'un même projet**. Fermer une tâche terminée et en ouvrir une nouvelle ne fait donc pas perdre le contexte utile — les décisions, conventions et l'état du travail restent accessibles. Tu n'as pas besoin de garder un fil ouvert "au cas où" pour ne pas avoir à tout réexpliquer.

À RETENIR

Le projet (dossier + instructions + mémoire) est fait pour durer des mois. La tâche (le fil) est faite pour durer le temps d'un objectif, puis se refermer.

3. Fragmenter : oui, mais par objectif — pas par message

La question n'est pas "court ou long", c'est "un sujet à la fois".

À éviter	À privilégier	Pourquoi
Un seul fil ouvert pendant des semaines pour tout le projet (contenu pédagogique, debug Godot, art, roadmap mélangés)	Une tâche = un objectif livrable (un lot de contenu, une fonctionnalité, un bug précis), refermée une fois faite	L'historique ne grossit pas indéfiniment ; chaque tâche reste focalisée et rapide à traiter
Ouvrir une nouvelle tâche à chaque micro-question sur un sujet en cours ("et ça ?", "et si...", "attends aussi...")	Rester dans la même tâche tant qu'on travaille sur le même objectif — le cache reste "chaud" et chaque échange coûte peu	Une nouvelle tâche repart de zéro sur le plan conversationnel : elle repaie le socle fixe et doit relire les fichiers concernés
S'acharner à corriger Claude plusieurs fois de suite sur le même point dans un fil déjà long	Après 2 corrections infructueuses, ouvrir une tâche neuve avec une consigne plus précise qui intègre ce qu'on vient d'apprendre	Un fil pollué par des tentatives ratées pousse Claude à répéter les mêmes erreurs ; un départ propre avec une meilleure consigne fait mieux, plus vite

4. Écrire des demandes efficaces

Une consigne précise limite l'exploration à l'aveugle — donc les allers-retours, donc la conso.

Vague	Précis	Effet
"Améliore le système de combat"	"Dans <code>combat.gd</code> , <code>take_damage()</code> ignore l'armure. Ajoute le calcul de réduction et un test qui vérifie 10 dégâts / 5 armure = 5 dégâts reçus."	Claude va droit au fichier concerné au lieu de scanner tout le projet
"Ajoute des questions de maths"	"Ajoute 50 questions de maths CE1 sur les tables de 2 et 5, format <code>csv/questions/ce1_math.csv</code> , ids suivants selon <code>ID_RANGES.md</code> ."	Pas besoin de deviner le format ni de relire tout l'historique pour retrouver la convention
"Corrige les accents"	"Corrige les accents manquants dans <code>lecture_cm1.csv</code> uniquement. Montre-moi le diff avant de valider."	Un périmètre limité + une vérification demandée = pas de relecture manuelle interminable dans le chat

UN RÉFLEXE QUI PAIE

Donne toujours un moyen de vérifier le résultat : un test, un diff, un nombre attendu, une capture d'écran. Sans ça, "ça a l'air fini" est le seul signal disponible, et c'est toi qui dois tout relire pour repérer l'erreur — ce qui redémarre des échanges supplémentaires.

5. Pour les gros morceaux : explorer, planifier, exécuter, vérifier

Sur une fonctionnalité qui touche plusieurs fichiers, ne pas foncer direct dans le code.

1

Explorer

Demander à Claude de lire et résumer l'existant avant de toucher à quoi que ce soit.

2

Planifier

Demander un plan : quels fichiers, quelle approche, quels cas limites. Le valider ou le corriger avant de lancer le code.

3

Exécuter

Laisser Claude implémenter en suivant le plan validé, fichier par fichier si le lot est gros.

4

Vérifier

Tester, lancer le jeu, comparer à une capture d'écran — avec une preuve, pas juste "c'est bon".

Ce détour évite le pire scénario côté conso : un développement lancé sur une mauvaise piste, qu'on corrige à coups de petites remarques pendant des dizaines de messages plutôt que de repartir sur un plan clair.

6. Pièges fréquents à surveiller

- **Le fil "fourre-tout"** — design de jeu, contenu pédagogique et debug Godot mélangés dans un seul fil qui dure des semaines.
- **La correction en boucle** — reformuler la même remarque trois fois dans le même fil au lieu de repartir avec une consigne plus précise.
- **Coller de gros blocs de texte ou de CSV directement dans le chat** plutôt que de laisser Claude lire le fichier depuis le dossier connecté.
- **Des instructions de projet trop longues ou obsolètes** — elles se rechargent à chaque tâche ; au-delà de l'essentiel, elles noient les règles importantes et gaspillent du contexte pour rien.
- **Laisser un fil inactif puis y revenir des jours après** — le cache est froid, le message suivant recharge tout à plein tarif.
- **Fragmenter à l'excès** — couper un travail cohérent en dix tâches séparées fait payer dix fois le "démarrage à froid" au lieu d'une fois.

7. Checklist de démarrage pour le prochain projet

- ☐ Créer un nouveau Projet Cowork dédié au nouveau jeu (pas dans le projet actuel)
- ☐ Connecter le dossier du nouveau projet dès le départ
- ☐ Écrire des instructions de projet courtes : version de Godot, conventions de code, ce qui doit être vérifié avant d'écrire du code — viser moins de 200 lignes
- ☐ Prévoir dès le début une structure de mémoire par sujet (comme pour le projet actuel : specs d'art, roadmap, conventions de contenu...)
- ☐ Une tâche = un objectif livrable ; la refermer une fois le résultat validé
- ☐ Donner un critère de vérification dans chaque demande de code (test, diff, capture, nombre attendu)
- ☐ Pour les grosses fonctionnalités : demander un plan avant d'implémenter

- Après deux corrections infructueuses sur le même point, ouvrir une tâche neuve plutôt que d'insister

Pense-bête — une page à garder sous la main

STRUCTURE

- 1 projet = 1 jeu (dossier + instructions + mémoire)
- 1 tâche = 1 objectif livrable, refermée une fois faite
- La mémoire du projet survit d'une tâche à l'autre — pas besoin de tout garder ouvert

DEMANDES

- Nommer le fichier, la fonction, le format attendu
- Donner un moyen de vérifier (test, diff, capture, nombre)
- Sur un gros morceau : demander un plan avant le code

SIGNAUX D'ALERTE

- Deux corrections ratées sur le même point → nouvelle tâche, consigne plus précise
- Le fil dure depuis des semaines et mélange plusieurs sujets → le refermer, répartir en tâches séparées
- Un fil inactif depuis plus d'une heure → s'attendre à un rechargement complet au prochain message

Sources : documentation officielle Anthropic — « Best practices for Claude Code », « Manage costs effectively », « Effective context engineering for AI agents », « Best practices for getting started with Claude Cowork », « Organize your tasks with projects in Claude Cowork ».